

Accelerating FHE-based PIR via GPU

Guangbei Yi*, Yu-Te Ku†, Minzhang Li*, Shih-Hao Hung†, Feng-hao Liu*

*Washington State University: {guangbei.yi, minzhang.li, fenghao.liu}@wsu.edu

†National Taiwan University: d08946006@g.ntu.edu.tw, hungsh@csie.ntu.edu.tw

Abstract—Private Information Retrieval (PIR) enables a client to retrieve an item from a database without revealing the query index. PIR has numerous applications, including anonymous messaging, private contact tracing, and safe browsing. Recent constructions based on Fully Homomorphic Encryption (FHE) significantly reduce communication costs and improve computational efficiency. Because these schemes rely on lattice-based cryptography, they also offer promising post-quantum security.

This work investigates hardware acceleration for FHE-based PIR, motivated by emerging advances in GPU-accelerated FHE computation. We identify the key performance-critical components within FHE-based PIR pipelines and design GPU-optimized implementations tailored for these workloads. By integrating these optimizations into end-to-end PIR protocols, we achieve substantial reductions in query latency. We provide comprehensive benchmarks demonstrating the performance gains offered by GPU acceleration.

Index Terms—Private Information Retrieval, Fully Homomorphic Encryption, GPU Acceleration

I. INTRODUCTION

Private Information Retrieval (PIR) is a foundational cryptographic primitive that enables a client to retrieve a specific record from a server-hosted database without revealing the queried index to the server. Since its first proposal by Chor et al. [1], PIR has evolved from a purely theoretical construct into a practically deployed building block for privacy-preserving applications, including but not limited to anonymous communication [2]–[4], location privacy [5], and safe browsing services [6], [7].

The PIR literature spans a wide range of settings and design dimensions. Existing schemes are commonly categorized into multi-server [7]–[9] versus single-server [2], [6], [10]–[16] constructions, stateful [6], [12], [14], [16]–[18] versus stateless [19]–[21] protocols, and several other orthogonal classifications. From a technical standpoint, approaches range from computation-intensive designs based on Fully Homomorphic Encryption (FHE) [6], [14], [19]–[21] to lightweight solutions relying solely on Pseudo-Random Functions (PRFs) [22], [23].

Early FHE-based PIR schemes were long regarded as theoretically sound but practically inefficient. This perception has changed substantially in recent years [2], [6], [14], [24], driven by advances in homomorphic computation and optimizations tailored specifically to the PIR workload. In particular, recent single-server FHE-based schemes can be broadly divided into two paradigms: hint-heavy designs, which achieve low online latency at the cost of substantial client-side storage [6], and hintless designs, such as Spiral [14] and WhisPIR [19], which eliminate persistent client state but incur higher server-side computation. While hint-heavy

schemes typically offer better online performance and higher throughput, hintless schemes have seen significant efficiency improvements in recent years [25].

Our Focus. This work focuses on GPU acceleration of *KsPIR* [25], a state-of-the-art hintless single-server PIR construction. *KsPIR* introduces a novel dimension-folding realization and optimized Baby-step/Giant-step (BSGS) matrix–vector multiplication, resulting in three variants that trade varying amounts of offline preprocessing for faster online response times. Like many prior FHE-based PIR implementations [6], [14], [19]–[21], the original *KsPIR* implementation for practical evaluation is single-threaded and CPU-based.

Given that the core FHE operations and homomorphic computations in PIR workloads are highly parallelizable, GPU-based acceleration presents a natural and promising direction for improving the practicality and scalability of single-server PIR systems.

A. Our Contributions

We design a GPU accelerator for *KsPIR* [25], achieving substantial performance improvements over prior CPU-based implementations in terms of response latency and system throughput. Our key contributions are as follows:

- **GPU-accelerated design for KsPIR.** We implement a GPU-based Number Theoretic Transform (NTT) to accelerate FHE operations, where polynomial multiplication dominates the computational cost. Building on this, we optimize the entire homomorphic computation pipeline of *KsPIR* to fully exploit GPU parallelism. A central challenge lies in GPU memory management: FHE keys and the PIR database are too large to be fully resident in on-chip GPU memory. We therefore carefully design data layouts, streaming strategies, and memory-access patterns to minimize data movement across the GPU memory hierarchy and reduce CPU–GPU communication overhead, resulting in efficient end-to-end execution tailored specifically to the *KsPIR* workload.
- **Comprehensive experimental evaluation.** We conduct extensive experiments to evaluate the performance of our GPU-based *KsPIR* implementation. Our evaluation measures query response time, throughput, and scalability across different database sizes and parameter settings, and compares against CPU-based baselines. On an NVIDIA RTX 3090, we achieve 27.5–29.9× and about 20× offline speedups for Constructions 1 and 2, 82–97× speedup for

their shared online phase, and about 2.46 GB/s sustained throughput for Construction 3 through 16 GB.

II. BACKGROUND

We use the notations following [25]. We denote the cyclotomic ring as $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^n+1)$ where n is power-of-two. Let p and q be the plaintext and ciphertext moduli respectively, with $\Delta = \lfloor q/p \rfloor$ as the scaling factor. A ciphertext in the RLWE scheme under secret $s \in \mathcal{R}_q$ encrypting a message $\mu \in \mathcal{R}_p$ is defined as the pair $(b, a) \in \mathcal{R}_q^2$ such that $b = as + e + \Delta\mu$. We further utilize the RGSW encryption scheme and denote its external product with an RLWE ciphertext as $\square$. The function $\text{Ext}(\cdot)$ extracts an LWE ciphertext from the constant term of an RLWE polynomial. Transformations between the coefficient and transform domains are denoted by NTT and NTT^{-1} . The function $\text{coeff}(\cdot)$ returns the coefficient vector.

A. The KsPIR Protocol [25]

Luo et al. proposed three PIR schemes in [25].

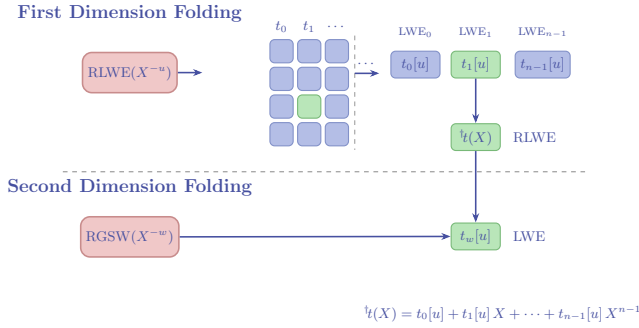


Fig. 1. A basic PIR protocol in two dimensions. Redraw from [25].

Construction 1: Two-Dimensional Dimension Folding.

The first construction encodes the database $\text{DB} \in \mathbb{Z}_p^{n \times n}$ by treating each column as the coefficient vector of a polynomial $t_i(X) = \sum_{j=0}^{n-1} \text{DB}[j, i]X^j$. To retrieve the record at index (u, w) , the client generates a query consisting of an RLWE encryption of the monomial X^{-u} and an RGSW encryption of X^{-w} . The server executes a "first dimension folding" by homomorphically extracting the u -th coefficients $t_0[u], \dots, t_{n-1}[u]$ and packing them into a single RLWE ciphertext $t(X) = \sum_{i=0}^{n-1} t_i[u]X^i$. The retrieval is finalized via a "second dimension folding" using the external product $(b', a') = \text{RGSW}_s(X^{-w}) \square t(X)$, followed by an $\text{Ext}(\cdot)$ operation to produce an LWE ciphertext encrypting the target entry $\text{DB}[u, w]$, as depicted in Figure 1. Although dimension folding technique is conceptually similar to that in Spiral [14], it additionally eliminates the need for expensive ciphertext expansion steps.

Construction 2: Offline-Optimized Response. The second construction builds upon the observation that an RLWE ciphertext (b, a) contains a message-independent component a that is uniformly sampled from $\mathcal{R}_q$. Since a does not depend on the specific query index u , the server can pre-calculate the

message-independent portions of the dimension folding transformations. Specifically, the heavy linear transformations and key-switching accumulations required for the first dimension folding are shifted to a database-dependent offline phase.

By pre-computing the partial RLWE component a' , the server reduces the online workload to processing only the message-dependent b component upon receiving the query. This strategy results in a substantial improvement in online throughput, achieving a response time approximately $10.7\times$ faster than Spiral [14], [25]. This design is particularly suitable for GPU acceleration, as the offline component a' can be pre-processed and resident on the device, significantly narrowing the online computation to a light-weighted reduction.

Construction 3: Throughput-Oriented BSGS. The third construction focuses on maximizing overall throughput without requiring the pre-processing overhead of Construction 2. The database is represented as an $n/2 \times n/2 \times 2$ matrix $\mathbf{M}$ in the NTT domain. The server evaluates the homomorphic matrix-vector product $\text{ans} = \text{BSGS}(\mathbf{M}, (b, a))$ using a highly optimized Baby-step Giant-step strategy. This operation extracts the desired column as an RLWE encryption of $\text{NTT}^{-1}(\mathbf{M} \cdot \mathbf{u})$, where $\mathbf{u}$ is the query unit vector.

This design is distinguished by its high rate (plaintext-to-response ratio) and its efficiency in handling large-scale databases where pre-processing may be impractical. By leveraging the NTT domain for matrix operations and utilizing optimized automorphism steps, Construction 3 achieves an overall throughput approximately $5.8\times$ faster than Spiral [14], [25]. For hardware acceleration, this construction provides a rich field of massive data parallelism within the nested loops of the BSGS algorithm, which we map directly to the GPU's SIMT architecture.

B. Core Operations in KsPIR

While Section II-A outlines the structural workflows of the KsPIR variants, evaluating their concrete efficiency requires decomposing these protocols into their underlying cryptographic primitives and memory constraints. Across all three constructions, the server-side workload is dominated by three atomic operations:

External Products. The external products are employed universally for the second dimension folding ($\mathbf{C} \square \text{ans}_0$) in all constructions in [25], which executes a single RGSW-by-RLWE ciphertext multiplication. And its accumulated form is the computational core of the offline phases in Constructions 1 and 2. Evaluating the external product $\square$ requires applying a gadget decomposition $g^{-1}(\cdot)$ to the RLWE input to produce l digits, followed by parallel NTT transformations and polynomial multiplications. Accumulating n such products translates to $O(n \cdot l)$ NTT conversions and multiply-add accumulations, rendering it an exceptionally compute-intensive task.

BSGS Matrix-Vector Multiplication. Construction 3 relies on homomorphic automorphisms (rotations) and plaintext-ciphertext slot-wise multiplications. Evaluating $\text{BSGS}(\mathbf{M}, (b, a))$ requires computing automorphisms on an RLWE ciphertext, followed by a key-switching step. These operations are performed entirely within the NTT domain to extract the final RLWE encryption of $\text{NTT}^{-1}(\mathbf{M} \cdot \mathbf{u})$.

Polynomial Transforms. Underpinning both the external products and the BSGS evaluations are the Number Theoretic Transform (NTT) and its inverse (NTT^{-1}). These transforms are ubiquitous across the KsPIR variants, as they are the prerequisite for efficiently computing any polynomial multiplication. By converting a polynomial $a \in \mathcal{R}_q$ from its coefficient representation into a point-value form $\tilde{a} = \text{NTT}(a)$, computationally expensive polynomial convolutions are reduced to linear, coefficient-wise Hadamard products $\tilde{c} = \tilde{a} \odot \tilde{b}$. Consequently, the overall computational workload of the server fundamentally scales with the sheer volume of NTT operations invoked during the two dimension folding stages.

III. GPU-ACCELERATED KsPIR: SYSTEM DESIGN

The original KsPIR implementation [26] is primarily optimized for single-core CPU execution, leaving significant opportunities for efficiency gains in a GPU environment. GPUs offer massive parallelism that can be effectively leveraged across several core operations of KsPIR, including high-dimensional polynomial transforms, external products, and the repeated movement of large key material. In this section, we present a GPU-native execution paradigm for all three KsPIR constructions. In particular, we identify three core components that benefit most from GPU-oriented designs: the synchronization overhead of $\text{NTT}/\text{NTT}^{-1}$ operations, the arithmetic cost of external-product accumulation, and the memory pressure induced by large databases and key-switching procedures.

A. Problem Statement

The atomic operations outlined in Section II-B are sufficient to realize the KsPIR, yet executing them naively on conventional CPUs or unmodified GPU pipelines yields severe performance degradation. We identify the following hardware-level bottlenecks as the primary obstacles to an efficient implementation.

NTT/ NTT^{-1} Transform Bottleneck. On CPUs, the sequential memory access patterns across the $O(n \log n)$ butterfly stages inevitably lead to severe cache thrashing for large ring dimensions. On GPUs, a direct implementation of the butterfly network exposes abundant parallelism within each stage, but it also creates a global-memory boundary between dependent stages. Each stage writes intermediate coefficients that the next stage must read, and the arithmetic in a single butterfly stage is too small to hide this traffic. As a result, the transform can become latency-bound even when many CUDA cores are available.

External Product Overhead. On multi-core CPUs, the sequential throughput of these decompositions and polynomial multiplications constrained by the CPU's limited arithmetic density and finite SIMD width. Conversely, a naive GPU port typically maps each logical sub-step to separate kernel launches. This approach forces the system to materialize intermediate digit vectors in global memory. The resulting execution path is heavily "launch-bound": the kernel scheduling overhead and repeated round-trips to global memory entirely eclipse the actual arithmetic execution time.

The Memory Wall in Accumulation. Evaluating the large-scale summations of external products required by Constructions 1 and 2 ($\text{hint}_1 = \sum_{i \in [n]} \text{pk}_i \boxtimes v_i$ and $\text{hint}_2 = \sum_{i \in [n]} \text{KSkey}_i \boxtimes a[i]$), triggers a large memory bottleneck. Because the associated key (pk or KSkey) is orders of magnitude larger than the per-query ciphertexts which comprises n RGSW ciphertexts approximately 768 MB for $n = 4096$ and $l = 3$, the overhead of repeatedly uploading these long-lived objects or rematerializing them in global memory can easily dominate the total online response time.

The engineering challenge is therefore twofold: the arithmetic path must keep intermediate data close to the CUDA cores, and the memory system must avoid paying host-device transfer costs for long-lived protocol objects on every query.

Execution units and notation. We use a *database chunk* as the unit mapped to one independent PIR subproblem before any final packing step. For Constructions 1 and 2, one chunk is an $n \times n$ coefficient matrix, matching the notation in Section II-A. We use the following size notation throughout this section. Let w_q be the number of bytes used to store one coefficient modulo q . The device footprint of one database chunk is denoted by $S_{\text{DB}} = n^2 w_q$, so a database with r chunks occupies $r S_{\text{DB}}$. For Construction 1, $S_{\text{pk}} = \Theta(\ln^2 w_q)$ denotes the public-key material kept by the server. For Construction 2, $S_{\text{ks}} = \Theta(\ln^2 w_q)$ denotes one per-chunk key-switching table. When such a table is streamed, β denotes the column-block size and $S_{\text{blk}} = 2l\beta n w_q$ denotes the size of one streamed key block.

The rest of this section turns these observations into three implementation techniques: fused multi-block NTT, FMA-accelerated external products, and a load-compute pipeline for large resident objects.

B. Fused Multi-block NTT

The transform bottleneck in Section III-A comes from the global-memory boundary introduced by a stage-by-stage GPU schedule. We therefore use a fused execution schedule. We partition the butterfly network at a tile size τ : stages whose butterfly group fits inside a tile execute in one shared-memory kernel, while larger-stride stages remain as global kernels. The butterfly portion is reduced from $\log_2 n$ stage launches to $\log_2(n/\tau) + 1$ launches, and the saved launches apply to every batched transform. Figure 2 illustrates the resulting schedule.

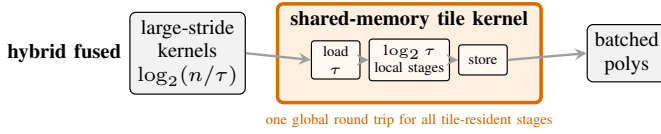


Fig. 2. Hybrid fused schedule for the multi-block NTT.

```

1 K ← split_beta(KSkey, β)
2 load_async(K[0], B0)
3 for j = 0..m-1:
4   if j+1 < m: load_async(K[j+1], B1); wait(B0)
5   FMA_inplace(acc, digits, B0); swap(B0, B1)

```

Fig. 3. View of the β -blocked fused FMA schedule.

Within the tile kernel, each block loads a contiguous τ -coefficient tile, performs the local butterfly stages in shared memory, and writes the tile back once. We keep the natural contiguous coefficient layout. We do not use warp-shuffle special cases in this transform, nor do we rely on padding or swizzling for bank-conflict avoidance, because those optimizations would require layout conversions across NTT, key-switching, and BSGS kernels. For this workload, avoiding extra global-memory traffic is more important than squeezing a small isolated improvement from one transform stage.

C. FMA-Accelerated External Product

After signed-digit decomposition, the expensive part of both external products and key switching is the same modular contraction between digit polynomials and key components. We implement this contraction as one fused modular FMA primitive:

$$\text{acc} \leftarrow \text{acc} \pm a \cdot b \pmod{q}.$$

The kernel uses a double-width intermediate and accumulates directly into the output ciphertext, removing the scratch vector and separate reduction pass used by a stage-by-stage implementation.

The same kernel serves the three constructions with different inputs. Construction 1 contracts decomposed query polynomials with public-key material. Construction 2 contracts the same query-side representation with precomputed key-switching blocks. Construction 3 invokes the primitive after its BSGS first fold. For Construction 2, the contraction is further organized into β -column blocks, so the same arithmetic kernel can run either on resident key-switching tables or on streamed key blocks. The row-parallel database reduction used by Constructions 1 and 2 maps rows to thread blocks and stays unchanged across the resident and streamed paths.

Figure 3 summarizes the blocked FMA schedule that connects the fused arithmetic kernel with the streaming path used by Construction 2. The next key block is requested before the current block enters the in-place FMA, so the same kernel can be reused while host-device transfer and modular accumulation overlap.

D. Load-Compute Pipeline Design

The memory path uses GPU residency tiers together with construction-aware dispatch. The dispatch policy binds residency and streaming decisions to the footprint of each KsPIR

```

1 load_async(DB[0], B0)
2 for c = 0..r-1:
3   wait(Bcur); y ← BSGS(Bcur)
4   if c+1 < r: load_async(DB[c+1], Bnext) // preload
5   z ← ExtProd(y); out ← Pack(out, z)
6   swap(Bcur, Bnext)

```

Fig. 4. Chunk prefetch schedule.

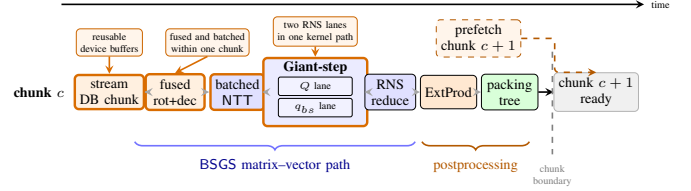


Fig. 5. Construction 3 per-chunk execution path with key GPU scheduling choices annotated.

construction. For Construction 1, the dispatcher uses the resident path when $S_{pk} + rS_{DB}$ fits in the device-memory budget. For Construction 2, full residency requires $r(S_{DB} + S_{ks})$. When this does not fit, the key-switching table is preflattened into β -column blocks of size S_{blk} and moved through double-buffered pinned staging. In the steady state, the transfer stream issues the load for block $j+1$ while the compute stream contracts block j with the current accumulator, i.e., the online loop follows a $\text{load}(j+1) \parallel \text{compute}(j)$ schedule. Only the buffer handoff is synchronized, so most host-device traffic is hidden behind the fused FMA kernel. This is the main load-compute pipeline in our system.

Construction 3 uses chunking for a different reason. Its BSGS-packed database can be larger than the desired resident footprint, so the server processes one database chunk at a time through reusable device buffers. Within a chunk, baby-step rotations are fused with decomposition, batched NTTs prepare the rotated polynomials, and the giant-step phase accumulates over the diagonal-packed database. The CRT main modulus Q and auxiliary modulus q_{bs} are carried as two arithmetic lanes inside the same batched kernels, not as independent CUDA streams. After the BSGS output is produced, the external product and packing tree finish chunk c while the free buffer starts prefetching chunk $c+1$. The next chunk still enters the BSGS path only after the current chunk boundary, but its input transfer is moved off the critical path. Figure 4 shows where the preload is issued, and Figure 5 shows the corresponding execution path.

IV. EXPERIMENTAL EVALUATION

A. Experimental Setup

All experiments run on a single host with an NVIDIA RTX 3090 GPU (24 GB) and an AMD Threadripper 3970X CPU (16 worker threads), under Ubuntu 24.04.4 with CUDA 13.1. We use the KsPIR parameters $N = 4096$, $q_1 = 2^{50}$, $q_2 = 4398046504961$, $\ell = 3$, $B_g = 2^{17}$, and $p = 2^{16}$; database size is the logical plaintext footprint, with one base chunk occupying $N \times N \times 2 = 32$ MB. Reported latencies are medians over benchmark iterations, and throughput is database size divided by online latency.

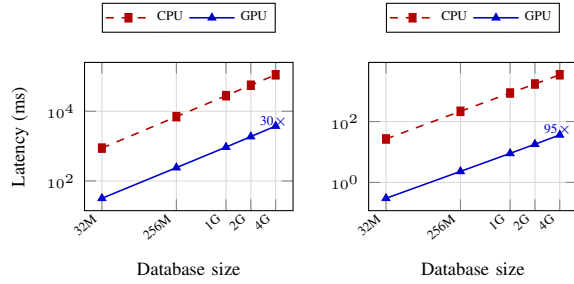


Fig. 6. Latency scaling for Constructions 1 and 2 (log-log). (a) Offline: 27.5–29.9 $\times$ speedup. (b) Online: 82–97 $\times$, sub-ms at 32 MB.

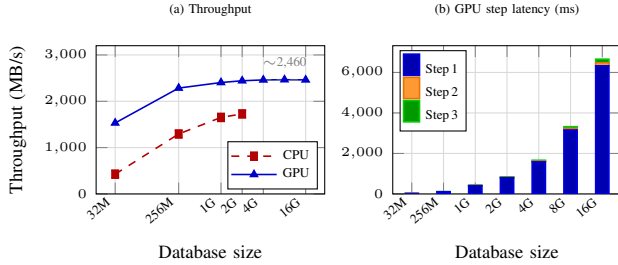


Fig. 7. Construction 3 (BSGS). (a) Throughput plateaus near 2,460 MB/s from 4 GB onward. (b) Step 1 dominates >95% of latency.

B. End-to-End Performance

a) *Key takeaway*: GPU acceleration fundamentally transforms KsPIR from latency-dominated CPU execution to throughput-oriented GPU execution: (1) offline preprocessing, which dominates CPU execution, becomes a minor fraction of total time on GPU; (2) online latency for Constructions 1 and 2 reaches sub-millisecond scale, making real-time single-query retrieval feasible; and (3) Construction 3 transitions from compute-bound scaling to throughput-saturated execution at approximately 2,460 MB/s, where further database growth adds latency linearly but no longer improves GPU utilization. The device-resident regime is constrained by key-switching table size (768 r MB for Construction 2), limiting Construction 2 scalability beyond $r = 8$ on the 24 GB RTX 3090.

Figures 6 and 7 present the detailed results.

b) *Offline speedup*: Construction 1 offline scales at **27.5–29.9 $\times$** from 32 MB through 4 GB (Figure 6a), reflecting linear scaling of batched transforms, row-wise products, and key-switching accumulation under bulk GPU parallelism. Construction 2 achieves $\sim$ **20 $\times$** offline speedup in the device-resident regime ($r \leq 8$); beyond that point, key-switching tables exceed the device memory budget and must be streamed.

c) *Online latency*: The shared online kernel achieves **82–97 $\times$** speedup (Figure 6b), reaching **0.3 ms** at 32 MB and **36.4 ms** at 4 GB. This near-linear scaling comes from the row-wise matrix–vector reduction, where each row maps to an independent thread block with minimal synchronization.

d) *Construction 3 throughput*: GPU throughput increases from 1,532 MB/s at 32 MB to approximately **2,460 MB/s** by 4 GB and remains flat through 16 GB (Figure 7a), with end-to-end speedups of **1.4–3.6 $\times$** . Figure 7b explains why these speedups are moderate: Step 1 (BSGS first-dimension processing) accounts for more than 95% of total latency, bounding

TABLE I
COMPARISON WITH CPU BASELINES (MS); CPU/GPU IS OUR SAME-HOST SPEEDUP.

Phase	DB	Paper CPU	Our CPU	Our GPU	CPU/GPU
<i>Construction 1 Offline</i>					
	36 MB	638	875	31.8	27.5$\times$
	256 MB	5,112	6,984	241	29.0$\times$
	1 GB	–	27,480	978	28.1$\times$
<i>Construction 2 Offline</i>					
	36 MB	93	262	13.2	19.8$\times$
	256 MB	746	2,103	101	20.8$\times$
	1 GB	2,955	8,396	1,043	8.1$\times$
<i>Shared Online (C1/C2)</i>					
	36 MB	14.5	26.8	0.3	89.3$\times$
	256 MB	133	218	2.3	94.8$\times$
	1 GB	694	869	9.0	96.6$\times$
<i>Construction 3 Online (BSGS)</i>					
	1 GB	–	620.0	425.8	1.5$\times$

the achievable end-to-end improvement. The marginal cost is 6.45 ms/chunk (GPU) versus 8.8 ms/chunk (CPU), giving a consistent **1.37 $\times$** marginal throughput advantage.

C. Scaling and Bottleneck Analysis

For Constructions 1 and 2, both offline and online latencies scale linearly with the number of database chunks r , and the GPU speedup factor remains nearly constant because each chunk maps to independent thread blocks. Construction 3 also scales linearly in latency, but throughput saturates once the per-chunk cost reaches the Step 1 floor. Step 1 combines automorphism, ciphertext reorientation, CRT-domain multiplication, and repeated key-switching, forming the long dependency chain visible in Figure 7b; Steps 2 and 3 together contribute less than 5%. Thus further Construction 3 gains require restructuring Step 1 through deeper fusion or finer pipeline overlap. For Constructions 1 and 2, the main scaling limit is memory: Construction 2’s key-switching tables require 768 r MB, limiting full residency to $r \leq 8$ on the 24 GB RTX 3090.

D. Validation of Performance Invariance

We also check three non-structural factors. Across Tranco Top-1M domain names [27], Open Food Facts records [28], and Simple English Wikipedia chunks [29], latency varies by less than 1% at matched (r, N) parameters, confirming that performance is determined by matrix dimensions rather than record content. Replacing our shared-memory NTT with a cuFFT-based double-precision FFT pipeline changes Construction 3 end-to-end latency by less than 0.3%, and per-query communication remains only 64–384 KB, negligible relative to server-side computation.

E. Comparison with Prior Work

Table I reports same-cost-class CPU/GPU speedups; C2 1 GB runs correctly in 1.04 s with pinned streaming. Construction 3 full scaling is in Figure 7.

V. DISCUSSION

A. Limitations and Future Directions

The main remaining bottleneck is algorithmic imbalance in Construction 3. Step 1 combines automorphism, ciphertext

reorientation, CRT-domain matrix–vector multiplication, and giant-step key-switching, and therefore dominates end-to-end latency even after GPU acceleration. The most promising next steps are deeper fusion inside Step 1, pre-transposed automorphism-key layouts for coalesced access, and double-buffered tile scheduling so decomposition and key switching overlap matrix multiplication.

The second limitation is memory scaling. Construction 2 requires 768r MB for key-switching tables, so full device residency ends once $r > 8$ on the 24 GB RTX 3090. Beyond that point, streaming now beats the paper CPU by about $2.8\times$ but remains transfer-bound and slower than resident mode; larger deployments should partition these tables across GPUs, compress or tile-pack key material, or stream table segments through the same double-buffered path used in our single-GPU prototype. Our largest Construction 3 CPU references also stop at 2 GB, and future evaluations should cover newer CPUs, newer GPUs, multi-GPU systems, and energy per query.

B. Construction Selection Guidelines

GPU acceleration sharpens the deployment trade-offs among the three constructions. Constructions 1 and 2 are best when minimum online latency is the goal; Construction 1 is operationally simpler, while Construction 2 is preferable only when its setup tables remain resident and can be amortized. Construction 3 is better for large databases and sustained throughput, but its speedup is bounded by Step 1. The practical choice is controlled by memory budget, latency target, and query volume; profiling shows 84–99% steady-state GPU time.

VI. CONCLUSION

We presented a GPU-native execution model for KsPIR based on fused multi-block NTT, fused modular FMA for external products, and load-compute pipelines for large key and database objects. On an RTX 3090, this design gives up to $30\times$ Construction 1 offline speedup, up to $21\times$ Construction 2 offline speedup in the device-resident regime, and up to $97\times$ speedup for their shared online phase. For Construction 3, it reaches $1.4\text{--}3.6\times$ end-to-end speedups and sustains about 2,460 MB/s from 4 GB to 16 GB. These results show that GPU acceleration can make FHE-based single-server PIR practical for lower-latency and higher-throughput deployments.

ACKNOWLEDGMENT

This research was supported in part by the National Science Foundation (NSF) under Grant CNS-2402031 and by the National Science and Technology Council (NSTC), Taiwan, under Grants 112-2221-E-002-159-MY3, 115-2119-M-002-006- and 113-2634-F-002-001-MBK.

REFERENCES

- [1] B. Chor, O. Goldreich, E. Kushilevitz, and M. Sudan, “Private information retrieval,” in *Proc. IEEE FOCS*, 1995, pp. 41–50.
- [2] S. Angel, H. Chen, K. Laine, and S. T. V. Setty, “PIR with compressed queries and amortized query processing,” in *Proc. IEEE S&P*, 2018, pp. 962–979.
- [3] A. Kwon, D. Lazar, S. Devadas, and B. Ford, “Riffle: An efficient communication system with strong anonymity,” *popets*, vol. 2016, no. 2, pp. 115–134, 2016.
- [4] P. Mittal, F. G. Olumofin, C. Troncoso, N. Borisov, and I. Goldberg, “PIR-tor: Scalable anonymous communication using private information retrieval,” in *Proc. USENIX Security*, 2011.
- [5] E. Fung, G. Kellaris, and D. Papadias, “Combining differential privacy and pir for efficient strong location privacy,” in *International Symposium on Spatial and Temporal Databases*. Springer, 2015, pp. 295–312.
- [6] A. Henzinger, M. M. Hong, H. Corrigan-Gibbs, S. Meiklejohn, and V. Vaikuntanathan, “One server for the price of two: Simple and fast single-server private information retrieval,” in *Proc. USENIX Security*, 2023, pp. 3889–3905.
- [7] D. Kogan and H. Corrigan-Gibbs, “Private blacklist lookups with checklist,” in *Proc. USENIX Security*, 2021, pp. 875–892.
- [8] H. Corrigan-Gibbs and D. Kogan, “Private information retrieval with sublinear online time,” in *Proc. EUROCRYPT*, 2020, pp. 44–75.
- [9] Z. Dvir and S. Gopi, “2-server pir with subpolynomial communication,” *Journal of the ACM (JACM)*, vol. 63, no. 4, pp. 1–15, 2016.
- [10] I. Ahmad, Y. Yang, D. Agrawal, A. El Abbadi, and T. Gupta, “Addr: Metadata-private voice communication over fully untrusted infrastructure,” in *15th {USENIX} symposium on operating systems design and implementation ({OSDI} 21)*, 2021.
- [11] A. Ali, T. Lepoint, S. Patel, M. Raykova, P. Schoppmann, K. Seth, and K. Yeo, “Communication-computation trade-offs in PIR,” in *Proc. USENIX Security*, 2021, pp. 1811–1828.
- [12] H. Corrigan-Gibbs, A. Henzinger, and D. Kogan, “Single-server private information retrieval with sublinear amortized time,” in *Proc. EUROCRYPT*, 2022, pp. 3–33.
- [13] E. Kushilevitz and R. Ostrovsky, “Replication is NOT needed: SINGLE database, computationally-private information retrieval,” in *Proc. IEEE FOCS*, 1997, pp. 364–373.
- [14] S. J. Menon and D. J. Wu, “SPIRAL: Fast, high-rate single-server PIR via FHE composition,” in *Proc. IEEE S&P*, 2022, pp. 930–947.
- [15] M. H. Mughees, H. Chen, and L. Ren, “OnionPIR: Response efficient single-server PIR,” in *Proc. ACM CCS*, 2021, pp. 2292–2306.
- [16] M. Zhou, A. Park, W. Zheng, and E. Shi, “Piano: Extremely simple, single-server PIR with sublinear server computation,” in *Proc. IEEE S&P*, 2024, pp. 4296–4314.
- [17] A. Lazzaretti and C. Papamanthou, “TreePIR: Sublinear-time and polylog-bandwidth private information retrieval from DDH,” in *Proc. CRYPTO*, 2023, pp. 284–314.
- [18] S. Patel, G. Persiano, and K. Yeo, “Private stateful information retrieval,” in *Proc. ACM CCS*, 2018, pp. 1002–1019.
- [19] L. de Castro, K. Lewi, and E. Suh, “Whisper: Stateless private information retrieval with low communication,” *Cryptology ePrint Archive*, 2024.
- [20] B. Li, D. Micciancio, M. Raykova, and M. Schultz, “Hintless single-server private information retrieval,” in *Proc. CRYPTO*, 2024, pp. 183–217.
- [21] S. J. Menon and D. J. Wu, “YPIR: High-throughput single-server PIR with silent preprocessing,” in *Proc. USENIX Security*, 2024.
- [22] A. Ghoshal, M. Zhou, and E. Shi, “Efficient pre-processing PIR without public-key cryptography,” in *Proc. EUROCRYPT*, 2024, pp. 210–240.
- [23] A. Ghoshal, M. Zhou, E. Shi, and B. Peng, “Pseudorandom functions with weak programming privacy and applications to private information retrieval,” in *Proc. EUROCRYPT*, 2025, pp. 284–313.
- [24] C. Aguilar-Melchor, J. Barrier, L. Fousse, and M.-O. Killijian, “XPIR: Private information retrieval for everyone,” *popets*, vol. 2016, no. 2, pp. 155–174, 2016.
- [25] M. Luo, F.-H. Liu, and H. Wang, “Faster FHE-based single-server private information retrieval,” in *Proc. ACM CCS*, 2024, pp. 1405–1419.
- [26] M. Luo, “kspir: Implementation of faster fhe-based single-server private information retrieval,” <https://github.com/mmmingluo/kspir>, 2024, accessed: 2026-04-30.
- [27] V. Le Pochat, T. van Goethem, S. Tajalizadehkhoob, M. Korczynski, and W. Joosen, “Tranco: A research-oriented top sites ranking hardened against manipulation,” in *Proc. NDSS*, 2019.
- [28] Open Food Facts, “Open Food Facts product database,” <https://world.openfoodfacts.org/data>, 2026, accessed: 2026-04-30.
- [29] Wikimedia Foundation, “Wikimedia Downloads,” <https://dumps.wikimedia.org/>, 2026, accessed: 2026-04-30.